

of a natural number is also defined as a natural number. Thus 1 is a natural number because 0 is defined as a natural number, and so on and so forth. This definition is based on the Peano axioms for natural numbers. To keep things simple, I will refrain from introducing any technical terms in the rest of our discussion. Further, it is beyond the scope of this article to discuss even simple Coq programs proving very basic theorems like De Morgan's laws. Therefore, we conclude our discussion of Coq by highlighting its main components and the strategy used in giving proofs using Coq.

Coq involves two types of commands: 'vernacular' and 'tactics'. Vernacular commands allow interaction with the Coq environment and include commands like *Theorem*, *Lemma*, *Fact*, etc. Tactics, on the other hand, are used to interact with the proof state and guide the proof process. They apply various proof strategies, such as simplifying expressions, applying logical rules, performing case analysis, and introducing hypotheses. Examples of tactical commands include *intros*, *rewrite*, and *destruct*. Coq also provides a specification language called Gallina. Interestingly, the word 'Coq' means 'rooster' in French, while 'Gallina' means 'hen' in Latin. Additionally, Coq has a built-in tactic scripting language called Ltac, which automates and customises the proof process. While Ltac is widely used with Coq, it is not often considered a standard part of Coq.

Now, let us discuss the strategy used in Coq to prove a theorem. Coq provides constructive proofs. It has a type called 'True', which consists of the set of all provably true propositions. Thus, in Coq, if you can find a value for a type, then it is considered proved. For example, the type 'nat', as seen in the line of code discussed earlier, proves that natural numbers do exist. The proof strategy in Coq follows this approach: a required theorem to be proven is denoted as a type. A proof for this theorem involves deriving a value for this type based on fundamental facts. This proof strategy also ensures that a false value can never be derived in Coq. The reason is that a value of 'False' is impossible to construct in Coq.

Lean

Now, let us turn our attention to Lean. Nowadays, we frequently encounter the names of companies known for developing proprietary software even when discussing free and open source software. Therefore, it comes as no surprise that Lean, the second interactive theorem prover we are discussing in this article, is developed by Microsoft Research. The project, primarily developed by Leonardo de Moura, was first released in 2013. Lean is free and open source software licensed under the Apache License 2.0. Figure 6 shows the logo of Lean.

At present, there are two versions of Lean available: Lean 3 and Lean 4. However, it's important to note that Lean 4 is not compatible with Lean 3, meaning that code written for

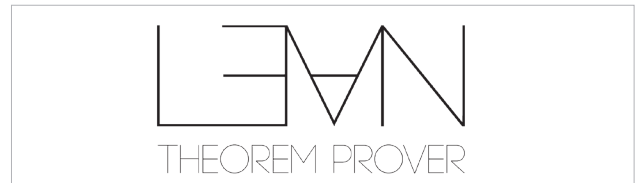


Figure 6: The logo of Lean

one version may not work in the other. One of the standout features of Lean is its powerful mathematical library known as *mathlib*. Initially, Lean 4 encountered issues when utilising this library, but these have now been resolved, and Lean 4 can seamlessly use *mathlib* as well. If you prefer to use Lean 4 without installing it in your system, you have the option of utilising a web-based environment accessible at <https://lean.math.hhu.de>. However, should you decide to install and run Lean 4 on your local machine, the most straightforward approach is to use the Visual Studio Code editor (VS Code editor). Although I won't delve into the detailed steps for setting up Lean 4 in VS Code editor here due to space constraints, I assure you that the installation and usage process is remarkably easy and straightforward.

Now, let us explore how Lean 4 code can be executed in both the VS Code editor and the web-based environment. Take a look at the line of code `#check 5 + 5`. Notice that Lean programs are saved with a '.lean' extension. When you run this code, it will produce the output `5 + 5 : Nat`, confirming that the number 10 is also a natural number.

It is time to wind up this article, but before we conclude our discussion, two important questions need to be addressed. First, are there any notable distinctions between Coq and Lean in their approach to obtaining a proof? I have been informed that Lean is better suited while using classical logic to derive a proof. However, Coq has a larger and older community around it when compared to Lean. Nevertheless, any disparities between the two will diminish over time since both software packages are experiencing rapid development. Second, are there any other applications for interactive theorem provers aside from proving theorems? Absolutely! Both Coq and Lean can be used to provide guarantees for the programs you have developed. Rather than relying on test cases to verify a program's correctness, you can utilise Coq or Lean to formally prove that the program is free of bugs. **END** 🐧

Acknowledgment:

I thank Julin Shaji, a former student of mine, for teaching me the basics of Coq. While preparing this article, I have referred to his notes (a work in progress) on Coq.

By: Dr Deepu Benson

The author is a free software enthusiast and his area of interest is theoretical computer science. He maintains a technical blog.